

FRANKENSTEIN

Meta-Agentic System for
Autonomous Agent Construction

PS-03 Hackathon Project Report

May 2026

One conversation. Working agents. Tested and ready.

Table of Contents

1. Problem Statement	3
2. Solution Overview	4
3. Unique Selling Proposition (USP)	5
4. System Architecture	6
5. Pipeline Deep Dive	8
6. Technology Stack	12
7. Data Models & State Management	13
8. Chroma Vector Memory Architecture	14
9. Frontend Architecture	15
10. Real-Time Streaming Architecture	16
11. Demo Targets & Validation	17
12. Challenges Solved	18
13. Scope & Limitations	19
14. Conclusion	19

1. Problem Statement

Building AI agents is not a coding problem -- it is a decision problem. For every natural-language requirement, an engineer must make fifty or more interdependent technical decisions: how many sub-agents, what roles each plays, which tools to bind, what reasoning strategy to use, how to structure memory, how to handle failures, what architecture pattern fits the workflow.

The tools and LLMs exist. The gap is not technology -- it is the translation layer between human intent and working agent architecture.

Today, that translation is done manually by engineers who understand prompt design, tool orchestration, and agentic patterns. This concentrates AI agent development among a small number of practitioners and makes every new agent a weeks-long engineering effort.

The Core Bottleneck

There are more AI agent use cases than there are engineers to build them. A loan officer who wants an AI assistant to review applications must explain the process to an engineering team. That team spends weeks learning the domain, picking tools, writing code, testing, and fixing. A procurement manager wanting AI to score suppliers goes through the same cycle. Every new agent is a custom engineering project. It takes too long, costs too much, and most teams don't have the engineers to spare.

The Question

Can an AI system take over that translation entirely -- not by simplifying it, but by automating the decision-making itself?

2. Solution Overview

Frankenstein is a meta-agentic system: an AI system that builds other AI agent systems. The human describes a problem in plain language. Frankenstein asks the right questions to extract what it needs, designs the architecture, stress-tests it, builds it, runs it, and fixes what breaks.

Building an agent requires two kinds of knowledge: domain knowledge (what the agent should do) and engineering knowledge (how to build it). Today, one person needs both. Frankenstein splits that burden -- the human provides domain expertise, Frankenstein provides the engineering.

Six-Stage Pipeline

Frankenstein runs a six-stage LangGraph StateGraph pipeline:

- | | |
|---|---|
| 1. Elicitor (gpt-4o-mini) | - Extracts domain knowledge from human via structured Q&A across 5 categories |
| 2. Architect (claude-sonnet-4-5) | - Generates framework-agnostic specification with agent roles, tools, memory, and flow |
| 3. Critic (gpt-4o) | - Adversarial spec review using 6 attack vectors; different model family on purpose |
| 4. Builder (claude-sonnet-4-5) | - Compiles validated spec into CrewAI or LangGraph code via plan-generate-validate-repair |
| 5. Tester (gpt-4o-mini) | - Runs agents in Docker sandbox, traces failures to spec-level root causes |
| 6. Learner (gpt-4o-mini) | - Stores build outcomes, patterns, and anti-patterns in Chroma for future RAG |

Human-in-the-Loop

Two human checkpoints: after requirements extraction ("Is this what you meant?") and after spec + critique review ("Approve the blueprint?"). The human validates, never writes code. Implemented via LangGraph's native interrupt() mechanism -- the graph pauses, WebSocket pushes the payload to the frontend, human reviews, and Command(resume=...) continues.

3. Unique Selling Proposition (USP)

Not One-Shot Generation

Most approaches pass a prompt to an LLM and generate framework code. Frankenstein does not work this way. It runs the same process a senior engineering team would -- compressed into a multi-agent pipeline. A single LLM prompt can generate agent code, but it cannot:

- Ask the human what's missing from their requirements
- Validate that selected tools actually support the required data formats
- Attack its own design for failure modes before building
- Run the generated code and trace failures back to architectural decisions
- Learn from previous builds to improve future ones

The Specification Layer

The specification layer is the core innovation. It serves as both a trust mechanism -- users can inspect and validate the blueprint before anything is built -- and a quality gate that ensures depth in the output. Frankenstein does not trade power for ease. The interface is simple, but the output matches what an experienced engineer would produce.

Cross-Model Adversarial Review

The Architect (Claude Sonnet) and Critic (GPT-4o) deliberately use different model families. Cross-model adversarial review catches flaws that same-model review misses -- shared blind spots in one model family are exposed by the other.

Self-Improving System

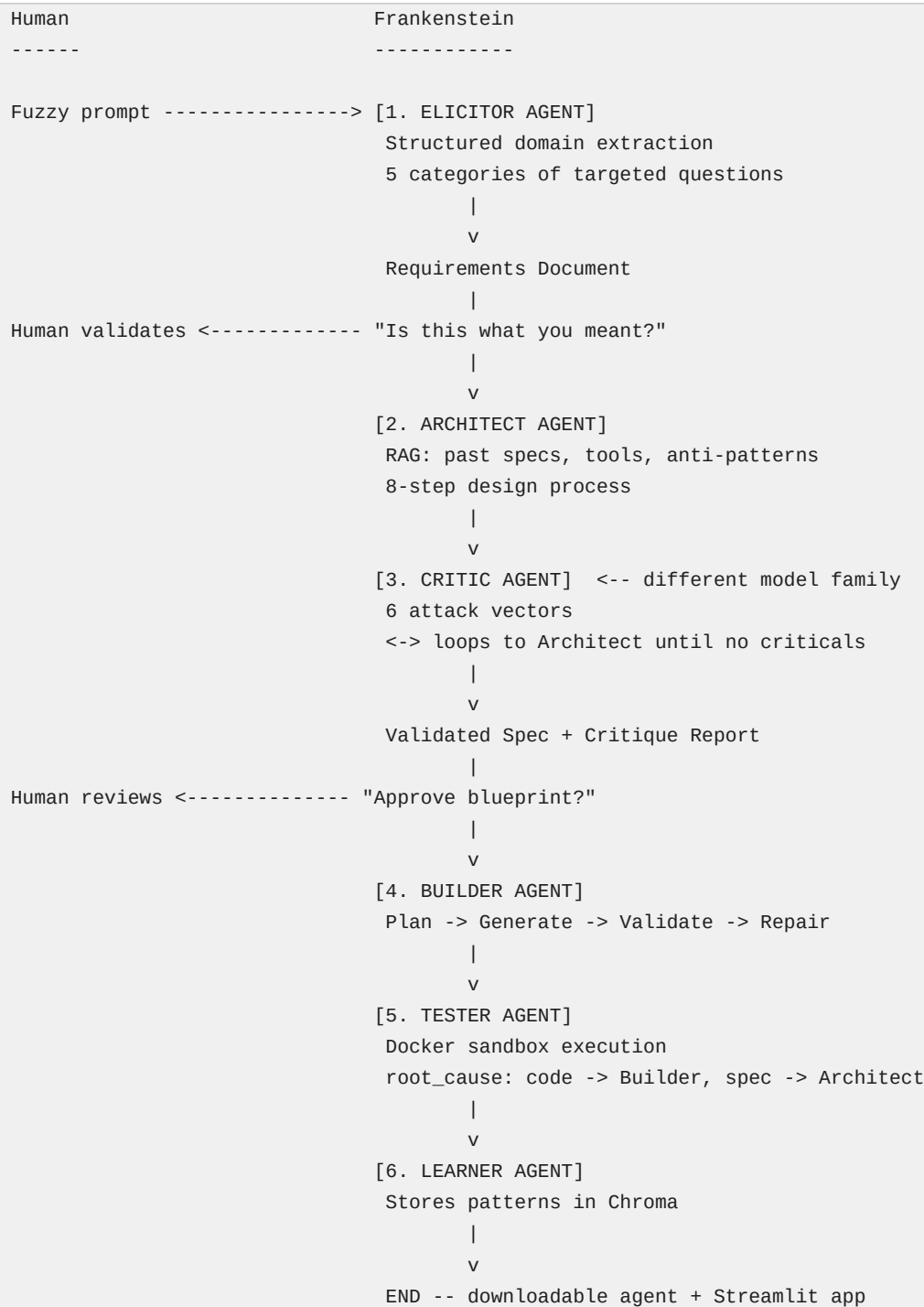
The Learner agent stores success patterns, failure patterns, and anti-patterns after every build. Future builds query this memory via RAG. The system literally gets smarter with every build.

Self-Healing Feedback Loops

Three feedback loops enable autonomous correction: Architect <-> Critic (spec revisions until no critical findings), Builder -> Validate -> Repair (AST/import validation with up to 3 repair cycles), and Tester -> Builder or Architect (failure traces classify root cause level -- code bugs go back to Builder, spec bugs go back to Architect).

4. System Architecture

High-Level Architecture Diagram



Pipeline Orchestration

Frankenstein itself is a LangGraph StateGraph. Each stage is a node. Edges define transitions. Conditional edges implement feedback loops. A single state object flows through the entire graph -- every node reads from it, does its work, writes back.

Conditional Routing

After Critique: if critical findings > 0 AND spec_iteration < max, route back to Architect. Otherwise, route to human review.

After Testing: if all tests pass, route to Learner. If failure is code-level, route to Builder. If failure is spec-level, route to Architect. If max iterations reached, route to Learner with partial success flag.

5. Pipeline Deep Dive

5.1 Elicitor Agent

Implementation: LangGraph subgraph with a loop. Runs gap analysis against 5 categories: Input/Output, Process, Data, Edge Cases, Quality Bar. Each category has a confidence score (0-1), loops until all ≥ 0.85 . Maximum 3 rounds of questions -- flags remaining gaps as assumptions. Uses gpt-4o-mini for fast, interactive Q&A.

Output: structured RequirementsDoc with typed fields including domain, inputs/outputs (DataSpec), process steps (ProcessStep), edge cases, quality criteria, and constraints.

5.2 Architect Agent

8-step process using claude-sonnet-4-5:

- Step 1: RAG Query -- query Chroma for similar past specs, tools, and anti-patterns
- Step 2: Task Decomposition -- break requirements into discrete computational tasks, tag each with capability type
- Step 3: Tool Selection -- match each task to a tool from the library by format compatibility
- Step 4: Agent Grouping -- related tasks grouped into same agent, independent chains become separate agents
- Step 5: Execution Flow -- analyze dependencies, pick pattern (sequential/parallel/hierarchical/graph)
- Step 6: Memory Design -- shared keys, strategy, persistence
- Step 7: I/O Contracts -- per-agent input/output schemas with typed fields
- Step 8: Pipeline I/O Contract -- how data enters and leaves the entire pipeline

The framework target is determined by flow design: graph patterns map to LangGraph, role delegation maps to CrewAI.

5.3 Critic Agent

Runs 6 attack vectors in parallel (5 programmatic + 1 LLM semantic), using gpt-4o:

Attack Vector	What It Checks
Circular Dependencies	Cycles in execution graph via topological sort
Format Compatibility	Agent A output matches Agent B input format
Dependency Completeness	Every agent receives all required fields from upstream
Dead-End Analysis	Agents that can fail without being caught
Resource Conflicts	Parallel agents writing same shared memory keys
Tool + Semantic Review	Tool capability match, coherence, reasoning strategy fit (LLM)

Output: CritiqueReport with findings scored as critical/warning/suggestion. Critical findings force Architect revision. Loop continues until no criticals remain (max 3 iterations).

5.4 Builder Agent

Multi-pass architecture using claude-sonnet-4-5:

- PLAN -- LLM produces a BuildPlan (tool signatures, agent task templates, kickoff inputs) before writing code
- GENERATE -- code conforming to the plan, framework-specific (CrewAI or LangGraph)
- VALIDATE -- `_validation.run_all()` with 8+ AST-level checks
- REPAIR -- if validation fails, send errors + RAG-retrieved past repair patterns to LLM for targeted fix (up to 3 cycles)

Validation System (8+ AST-Level Checks)

Code	What It Catches
SYNTAX_ERROR	File won't parse
UNRESOLVED_SYMBOL	Imports that don't exist
PARAM_SHADOWS_PYDANTIC	Params named schema, dict, json that break CrewAI
TOOL_WRONG_ARITY	Tool function doesn't follow <code>def fn(data: dict) -> dict</code>
TOOL_HAS_DEFAULTS	Default params break OpenAI strict schema
KICKOFF_EMPTY_INPUTS	<code>crew.kickoff(inputs={})</code> guaranteed to fail
ENTRY_POINT_MISSING_FIELD	<code>pipeline_input_schema</code> fields not wired
TOOL_NOT_DEFINED	Referenced in <code>agents.py</code> but never implemented

Generated Project Structure

```
generated_agent/  
+-- main.py           # entry point with env injection  
+-- agents.py         # agent definitions (roles, goals, backstories)  
+-- tools.py          # tool implementations (strict def fn(data:dict)->dict)  
+-- orchestration.py  # CrewAI crew or LangGraph graph definition  
+-- app.py            # Streamlit web UI  
+-- sample_data.json  # concrete test data matching pipeline_input_schema  
+-- requirements.txt   # Python dependencies  
+-- .env.example       # OPENROUTER_API_KEY=...  
+-- README.md         # prerequisites, setup, run command, I/O shape
```

5.5 Tester Agent

Execution pipeline using gpt-4o-mini:

- Generate test cases from spec I/O contracts
- Static validation via `_validation.run_all()` (same checks as Builder)
- Live execution in Docker sandbox with real LLM calls via OpenRouter
- Output validation: parse stdout JSON, flag known failure signatures
- Rule-based failure classification (regex) for common patterns; LLM handles the rest

Failure signatures detected: `EMPTY_INPUTS`, `MISSING_API_KEY`, `STRICT_SCHEMA`, `IMPORT_ERROR`, `KEY_ERROR`. Root cause routing: code-level loops to Builder, spec-level loops to Architect.

5.6 Learner Agent

Stores after every build:

- BuildOutcome with requirements hash, spec snapshot, framework, tools used, test results
- success_patterns -- what worked well
- failure_patterns -- what went wrong this build
- anti_patterns -- generalized patterns to avoid
- lessons_learned -- domain-specific takeaways
- repair_patterns -- error->fix pairs from Builder repair history

Updates tool compatible_with/incompatible_with from real build results.

6. Technology Stack

Full Stack Overview

Layer	Technology	Role
Pipeline	LangGraph StateGraph	Internal pipeline with conditional edges + SqliteSaver
Backend	FastAPI + Uvicorn	API server, WebSocket management, Docker orchestration
Frontend	React + Vite + Tailwind	Chat interface, spec review, code preview
3D Graphics	React Three Fiber	Landing page with custom GLSL shaders + bloom
LLM Routing	OpenRouter	Single API for all model access, model-per-agent
Vector DB	ChromaDB	5 collections, persistent, RAG retrieval
Execution	Docker	Pre-built base image, streaming execution
State Mgmt	Zustand	Frontend state management
Communication	WebSockets	3 channels: chat, status, preview
Generated Output	CrewAI / LangGraph	Framework-specific agent projects + Streamlit app

Model-per-Agent Strategy

Agent	Model	Reasoning
Elicitor	gpt-4o-mini	Fast, cheap, good at structured Q&A
Architect	claude-sonnet-4-5	Best at complex structured generation
Critic	gpt-4o	Different family -- cross-model review catches blind spots
Builder	claude-sonnet-4-5	Best at code generation with constraints
Tester	gpt-4o-mini	Fast analysis, rule-based does heavy lifting
Learner	gpt-4o-mini	Pattern extraction doesn't need frontier model

7. Data Models & State Management

Pipeline State Object

A single `FrankensteinState` (`TypedDict`) flows through the entire graph. Every node reads from it, does its work, writes back. Key fields include:

Elicitor Stage

`raw_prompt`, `session_id`, `elicitor_questions`, `human_answers`, `requirements` (`RequirementsDoc`), `requirements_approved`

Architect + Critic

`tool_library_matches`, `past_spec_matches`, `spec` (`AgentSpec`), `architect_reasoning`, `critique` (`CritiqueReport`), `spec_iteration`, `spec_approved`

Builder + Tester

`generated_code` (`CodeBundle`), `test_cases`, `test_results` (`TestReport`), `failure_traces`, `build_iteration`, `build_attempts`, `repair_history`

Learner

`build_outcome` (`BuildOutcome`)

Key Pydantic Models

- `RequirementsDoc` -- `domain`, `inputs/outputs` (`DataSpec`), `process_steps`, `edge_cases`, `quality_criteria`, `constraints`
- `AgentSpec` -- `metadata`, `agents` (with `roles/goals/tools/reasoning_strategy`), `tools`, `memory`, `execution_flow`, `error_handling`, `io_contracts`
- `CritiqueReport` -- `findings` (`vector`, `severity`, `description`, `location`, `evidence`, `suggested_fix`), `summary`
- `CodeBundle` -- generated project files with `agents.py`, `tools.py`, `orchestration.py`, `main.py`
- `TestReport` -- test results with `pass/fail` status per test case
- `FailureTrace` -- `test_name`, `error_type`, `raw_error`, `failing_agent`, `root_cause_level`, `root_cause_analysis`
- `BuildOutcome` -- `requirements_hash`, `spec_snapshot`, `framework_used`, `success/failure/anti` patterns

8. Chroma Vector Memory Architecture

5 Collections

Collection	Content	Used By
tool_schemas	14 pre-seeded validated tool definitions	Architect
spec_patterns	Past validated specs + outcomes	Architect
anti_patterns	Patterns that caused failures	Architect
domain_insights	Domain-specific learnings	Elicitor
builder_repair_patterns	Error->fix pairs from past repairs	Builder

14 Pre-Seeded Tool Schemas

Each tool schema includes: id, name, description, category, accepts/outputs formats, limitations, dependencies, code_template, and compatibility info.

Tool	Category
pdf_parser_pymupdf	Document extraction
ocr_tesseract	Document extraction
csv_parser	Data processing
json_transformer	Data processing
file_reader	Data processing
financial_calculator	Calculation
statistical_analyzer	Analysis
scoring_engine	Analysis
rule_engine	Reasoning
llm_reasoner	Reasoning
report_generator	Generation
data_visualizer	Generation
web_search	API call
code_executor	Execution

RAG Query Flow

When Architect receives requirements: (1) Query spec_patterns for similar past specs -- use successful ones as structural reference. (2) For each task, query tool_schemas -- select based on format chain compatibility. (3) Query anti_patterns to avoid known failure modes.

9. Frontend Architecture

Landing Page (React Three Fiber + Custom GLSL)

- LaboratoryVoid: custom GLSL shader background with biological noise patterns
- OrganismFragments: 6 3D shapes representing agents with custom vertex displacement shaders, per-fragment particle trails, and assembly animation
- EnergyConnections: animated THREE.Line objects connecting agents with traveling pulse dots
- AssemblyAnimation: fragments pull -> stitch -> lightning -> heartbeat sequence (Zustand-driven)
- Bloom post-processing via @react-three/postprocessing
- TypewriterHeadline: animated text reveal

Chat Interface Components

- Real-time dual WebSocket (chat + status channels)
- QuestionGroupCard: renders elicitor questions with per-category grouping, AI-assist button
- RequirementsCard: human checkpoint 1 -- approve/edit requirements
- SpecReviewCard: human checkpoint 2 -- approve/feedback on blueprint with critique findings
- TypingIndicator: 18 shuffled cycling facts about architecture while pipeline runs
- PipelineSidebar: live stage tracker with active/done/pending states
- ActivityPill: real-time sub-step updates
- CodingScreen: animated terminal showing build progress
- CompletionCard: download button + build stats (time, files, tests, agents)

Preview System

- CodeViewer: syntax-highlighted file explorer for generated code
- ExecutionPanel: Docker streaming execution with agent trace log
- StreamlitPreview: embedded iframe for generated Streamlit app (live preview)
- SplitPane: resizable panels for code + execution side-by-side

10. Real-Time Streaming Architecture

Queue-Based Node Streaming

Post-approval pipeline (builder->tester->learner) runs `graph.stream()` in a thread. A queue bridge forwards node events to the async WebSocket consumer in real-time. Supports cycling -- Builder appearing multiple times in repair loops gets retry-specific activity messages.

WebSocket Channels

Path	Purpose
/ws/chat/{session_id}	Bidirectional: user input, system messages, checkpoints
/ws/status/{session_id}	Server->client: stage updates, activity feed
/ws/preview/{session_id}	Server->client: Docker execution trace, run status

Docker Execution Environment

Pre-built base image (frankenstein-runner) with Python 3.11 + CrewAI + LangGraph + common packages pre-installed. Generated code mounted at runtime. Container killed after timeout (60s default). Output captured via Docker SDK for validation.

11. Demo Targets & Validation

PS-08: Loan Underwriting Co-Pilot

Multi-agent pipeline for document extraction, financial ratio calculation, risk scoring, compliance checking, and report generation. Tools: pdf_parser_pymupdf, ocr_tesseract, financial_calculator, rule_engine, report_generator, web_search.

PS-06: Supplier Reliability Scoring Agent

Data aggregation, statistical analysis, scoring, and ranking pipeline. Tools: csv_parser, statistical_analyzer, scoring_engine, data_visualizer, report_generator.

End-to-End Demo Flow

- 1. Open Frankenstein web app
- 2. Type prompt describing the desired agent system
- 3. Elicitor asks 3-5 targeted questions
- 4. Review and approve generated requirements (Checkpoint 1)
- 5. Review spec + critique report, approve blueprint (Checkpoint 2)
- 6. Watch build + test happen with real-time progress
- 7. Download working agent code + Streamlit app
- 8. Run it. It works.

Both demo targets run end-to-end: prompt -> elicitor questions -> requirements -> architect design -> critic review -> builder code -> tester execution -> learner storage -> downloadable project + Streamlit app.

12. Challenges Solved

Empty Elicitor Interrupt (Dead State Bug)

Elicitor round 2 generated 0 questions (all gaps scored ≥ 0.85) but still called `interrupt()` with empty payload. Frontend got empty question group, set `isWorking=false`, no loader, dead state. Fix: added early return with `elicitor_all_complete=True` when no questions generated.

WebSocket Disconnection at Builder Stage

Builder writes files to `generated_agents/`, uvicorn's `--reload` file watcher detects changes, restarts server, kills all WebSocket connections mid-pipeline. Fix: `--reload-exclude 'generated_agents/*' --reload-exclude 'chroma_data/*'`

Vite HMR WebSocket Collision

Broad `/ws` proxy in `vite.config.ts` caught Vite's own HMR WebSocket -> 'send before connect' errors. Fix: narrowed proxy to specific paths: `/ws/chat`, `/ws/status`, `/ws/preview`

Real-Time Streaming for Post-Approval Pipeline

Original implementation collected all stream events after completion, so no real-time updates during the longest part. Fix: queue-based bridge so `graph.stream()` pushes node events to async consumer for immediate WebSocket dispatch.

Builder Targeted Repair from Test Failures

Rebuilding after test failures regenerated everything from scratch, hitting the same validation errors. Fix: when previous code passed validation, Builder does targeted repair from failure traces instead of full regeneration.

Streamlit App Generation + Live Preview

Generated agents were CLI-only with no visual interface. Fix: Builder generates `app.py` (Streamlit web UI), `StreamlitService` launches it in Docker, frontend embeds via `iframe`.

13. Scope & Limitations

Clarity on what Frankenstein does not do:

- Does not understand domains autonomously -- relies on the human for domain knowledge. Frankenstein is the engineering team, not the product manager.
- Does not replace prompt engineering entirely -- the Elicitor improves input quality, but garbage domain knowledge in still produces garbage agents out.
- Does not guarantee perfect agents on first pass -- the test-and-fix loop exists because first builds will have issues. The system's strength is autonomous correction, not perfection.
- Does not work without a Tool Schema Library -- the Architect can only select tools it knows about. An empty library means no useful output.

14. Conclusion

Frankenstein demonstrates that the translation from human intent to working AI agent systems can be fully automated. By splitting the process into specialized agents -- each handling a distinct phase of the engineering workflow -- the system achieves what no single LLM call can: iterative requirements extraction, adversarial design review, template-driven code generation, sandboxed testing with root-cause analysis, and continuous learning.

The system collapses the expertise and time required to deploy agentic solutions from weeks to minutes. A domain expert in manufacturing, finance, or compliance can describe a workflow in plain language and receive a production-grade multi-agent system -- tested and ready.

By automating the full arc from intent to tested agents, Frankenstein removes the people bottleneck that currently limits AI agent adoption across industries.